

Compiling Typed APIs into Context-Free Languages

Breandan Considine, Clemente Pasti, Tim Vieira

August 12, 2026

Motivation: type-safe code completion

- ▶ Construct a $\ell_\Gamma = \mathcal{L}(G_\Gamma)$ consisting of well-typed statements
- ▶ No ill-typed statements $\sigma \in \ell_\Gamma$ or well-typed statements $\sigma \notin \ell_\Gamma$
- ▶ For any $\sigma \in \Sigma^*$ such that $\exists \tau. \sigma\tau \in \ell_\Gamma$, show τ 's in shortlex order

Motivation: type-safe code completion

- ▶ Construct a $\ell_\Gamma = \mathcal{L}(G_\Gamma)$ consisting of well-typed statements
- ▶ No ill-typed statements $\sigma \in \ell_\Gamma$ or well-typed statements $\sigma \notin \ell_\Gamma$
- ▶ For any $\sigma \in \Sigma^*$ such that $\exists \tau. \sigma\tau \in \ell_\Gamma$, show τ 's in shortlex order

```
1 #include <memory>
2 long* codes(); double* groups();
3 const char* text(); float* samples();
4 std::unique_ptr<int> fresh(), parse(const char*);
5 std::unique_ptr<int> average(float*), select(double&);
6 std::unique_ptr<int> combine(const char*, long*);
7 std::unique_ptr<int> schedule(const char*, long*, float*);
8
9 int main(int argc, char** args) {
10     std::unique_ptr<int> b[]
11 }
```

Motivation: type-safe code completion

- ▶ Construct a $\ell_\Gamma = \mathcal{L}(G_\Gamma)$ consisting of well-typed statements
- ▶ No ill-typed statements $\sigma \in \ell_\Gamma$ or well-typed statements $\sigma \notin \ell_\Gamma$
- ▶ For any $\sigma \in \Sigma^*$ such that $\exists \tau. \sigma \tau \in \ell_\Gamma$, show τ 's in shortlex order

```
1 #include <memory>
2 long* codes(); double* groups();
3 const char* text(); float* samples();
4 std::unique_ptr<int> fresh(), parse(const char*);
5 std::unique_ptr<int> average(float*), select(double&);
6 std::unique_ptr<int> combine(const char*, long*);
7 std::unique_ptr<int> schedule(const char*, long*, float*);
8
9 int main(int argc, char** args) {
10     std::unique_ptr<int> b□;
11 }
```

```
;
= fresh();
= average(samples());
= parse(text());
= select(groups()[argc]);
= combine(text(), codes());
= schedule(text(), codes(), samples());
```

Prior work on type-safe code completion

Legend  by construction  generation-time  post-generation  partial / empirical  absent / unclaimed

Lens	Type-directed search			Learned completion		Constrained decoding	
Feature	Perelman'12	Gvero'13	Pelsmaecker'22	Raychev'14	Semenkin'24	Melcer'24	Mündler'25
Language	C#	Scala	Statix DSL	Java	Py+JVM+JS	Python 3	TypeScript
Output unit	expression	expression	fragment	call sequence	full line	FIM span	function
Query format	partial expr	typed hole	cursor	holes	left prefix	FIM context	prompt
Core theory	bespoke	STLC	SDF3/Statix	bespoke	—	CSL quotient	prefix FSA
Ranking metric	type distance	usage priors	solver order	n-gram/RNN	LLM	LLM	LLM
Learned signal	—		—				
Filter timing	search	search	solver	rank only			
Syntax safe							
Type safe					—	—	
Statement-scale	—	—					
Parametricity	—	—		—	—		
Usable tool	VS Code	Eclipse	Spoofax	Slang	JetBrains	Git repo	Git repo
Public URL	—			—			

Gap Prior work tends to trade off probabilistic ranking, completion length, and type safety but does not guarantee exact enumeration over a ranked language of well-typed statements.

How long are typical source code statements?

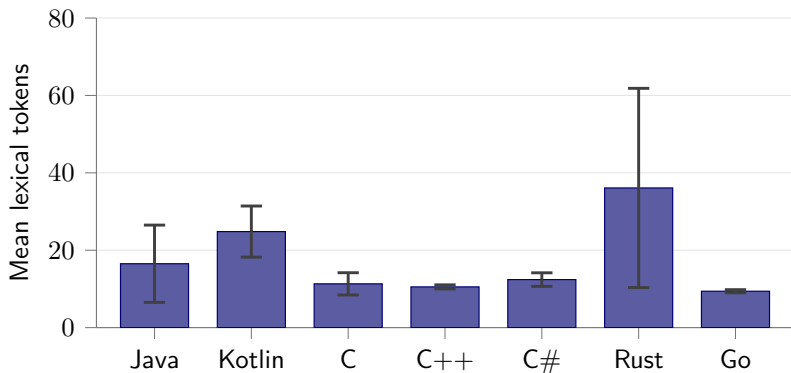


Figure: Whiskers are 95% CIs over sampled files. Statements are Tree-sitter statement nodes with comments and whitespace ignored. Samples are random GitHub files > 20kb; per-language sample sizes are shown below.

How large are programming language grammars?

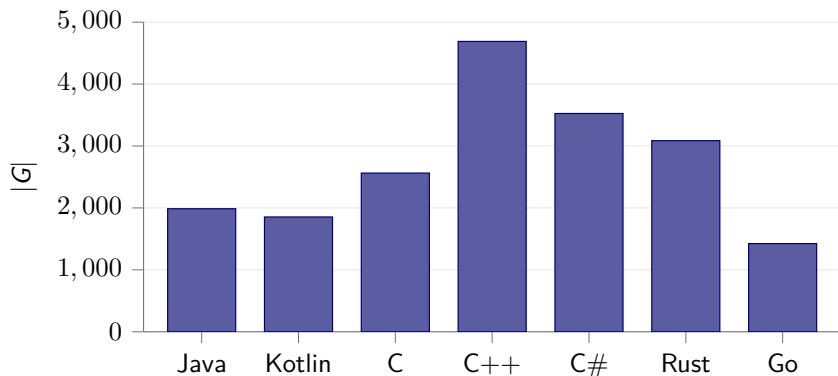


Figure: Grammar size is computed from the Tree-sitter reference CFGs, where $|G|$ is defined as $\sum_{w \in V} \sum_{(w \rightarrow \sigma) \in P} |w\sigma|$.

Correctness criterion: what must the CFG preserve?

Surface fragment G'

e.g., names, invocation, member chains

$S ::= E; \quad E ::= \text{ID} \mid \text{ID}() \mid \text{ID}(\text{ARGS}) \mid E.E \quad \text{ARGS} ::= E \mid E, \text{ARGS}$

Correctness criterion: what must the CFG preserve?

Surface fragment G'

e.g., names, invocation, member chains

$S ::= E; \quad E ::= \text{ID} \mid \text{ID}() \mid \text{ID}(\text{ARGS}) \mid E.E \quad \text{ARGS} ::= E \mid E, \text{ARGS}$

1. Surface projection π_Σ

erase names but preserve punctuation

$\pi_\Sigma(\mathbf{x.f(y)}) = \mathbf{ID.ID(ID)}$

checks that the shape belongs to G_0

Correctness criterion: what must the CFG preserve?

Surface fragment G'

e.g., names, invocation, member chains

$S ::= E; \quad E ::= \text{ID} \mid \text{ID}() \mid \text{ID}(\text{ARGS}) \mid E.E \quad \text{ARGS} ::= E \mid E, \text{ARGS}$

1. Surface projection π_Σ

erase names but preserve punctuation

$\pi_\Sigma(\mathbf{x.f(y)}) = \text{ID.ID(ID)}$

checks that the shape belongs to G_0

2. Typed projection Π_Γ

names $\rightarrow$ type slots; API names stay

$\Pi_\Gamma(\mathbf{x.f(y)}) \ni E\langle X \rangle.f(E\langle Y \rangle)$

contains all and only well-typed terms

Correctness criterion: what must the CFG preserve?

Surface fragment G'

e.g., names, invocation, member chains

$S ::= E; \quad E ::= \text{ID} \mid \text{ID}() \mid \text{ID}(\text{ARGS}) \mid E.E \quad \text{ARGS} ::= E \mid E, \text{ARGS}$

1. Surface projection π_Σ

erase names but preserve punctuation

$$\pi_\Sigma(\mathbf{x.f(y)}) = \text{ID.ID(ID)}$$

checks that the shape belongs to G_0

2. Typed projection Π_Γ

names $\rightarrow$ type slots; API names stay

$$\Pi_\Gamma(\mathbf{x.f(y)}) \ni E\langle X \rangle.f(E\langle Y \rangle)$$

contains all and only well-typed terms

Correctness criterion

For a target language L with surface fragment G' , construct G_Γ satisfying

$$\pi_\Sigma(\sigma) \in \mathcal{L}(G') \iff \Gamma \vdash \Pi_\Gamma(\sigma) \subset \mathcal{L}(G_\Gamma)$$

for all and only source statements σ accepted by a reference compiler for L .

Grammar modularization

C++ context

```
1 // Filetype: *.cpp
2 #include <string>
3 #include <vector>
4 #include "org/fizz.hpp"
5 #include "com/buzz.hpp"
6 using std::string;
7 // ...
8 int fizz() { return 3; }
9 string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12     string bang = "...";
13     □
14 }
```

Grammar contribution

$G_{C++} \cup$

Grammar modularization

C++ context

```
1 // Filetype: *.cpp
2 #include <string>
3 #include <vector>
4 #include "org/fizz.hpp"
5 #include "com/buzz.hpp"
6 using std::string;
7 // ...
8 int fizz() { return 3; }
9 string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12     string bang = "...";
13     
14 }
```

Grammar contribution

$G_{\text{C++}} \cup$

$G_{\text{<string>}} \cup$

Grammar modularization

C++ context

```
1 // Filetype: *.cpp
2 #include <string>
3 #include <vector>
4 #include "org/fizz.hpp"
5 #include "com/buzz.hpp"
6 using std::string;
7 // ...
8 int fizz() { return 3; }
9 string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12     string bang = "...";
13     □
14 }
```

Grammar contribution

$G_{C++} \cup$

$G_{\langle \text{string} \rangle} \cup$

$G_{\langle \text{vector} \rangle} \cup$

Grammar modularization

C++ context

```
1 // Filetype: *.cpp
2 #include <string>
3 #include <vector>
4 #include "org/fizz.hpp"
5 #include "com/buzz.hpp"
6 using std::string;
7 // ...
8 int fizz() { return 3; }
9 string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12     string bang = "...";
13     □
14 }
```

Grammar contribution

$$G_{\text{C++}} \cup$$
$$G_{\text{<string>}} \cup$$
$$G_{\text{<vector>}} \cup$$
$$G_{\text{org/fizz.hpp}} \cup$$

Grammar modularization

C++ context

```
1 // Filetype: *.cpp
2 #include <string>
3 #include <vector>
4 #include "org/fizz.hpp"
5 #include "com/buzz.hpp"
6 using std::string;
7 // ...
8 int fizz() { return 3; }
9 string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12     string bang = "...";
13     □
14 }
```

Grammar contribution

$$G_{C++} \cup$$
$$G_{<string>} \cup$$
$$G_{<vector>} \cup$$
$$G_{org/fizz.hpp} \cup$$
$$G_{com/buzz.hpp} \cup$$

Grammar modularization

C++ context

```
1 // Filetype: *.cpp
2 #include <string>
3 #include <vector>
4 #include "org/fizz.hpp"
5 #include "com/buzz.hpp"
6 using std::string;
7 // ...
8 int fizz() { return 3; }
9 string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12     string bang = "...";
13     □
14 }
```

Grammar contribution

$$\begin{aligned} &G_{\text{C++}} \cup \\ &G_{\text{<string>}} \cup \\ &G_{\text{<vector>}} \cup \\ &G_{\text{org/fizz.hpp}} \cup \\ &G_{\text{com/buzz.hpp}} \cup \\ &\{ \text{Type} \rightarrow \text{string} \} \cup \end{aligned}$$

Grammar modularization

C++ context

```
1 // Filetype: *.cpp
2 #include <string>
3 #include <vector>
4 #include "org/fizz.hpp"
5 #include "com/buzz.hpp"
6 using std::string;
7 // ...
8 int fizz() { return 3; }
9 string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12     string bang = "...";
13     □
14 }
```

Grammar contribution

$G_{C++} \cup$
 $G_{<string>} \cup$
 $G_{<vector>} \cup$
 $G_{org/fizz.hpp} \cup$
 $G_{com/buzz.hpp} \cup$
 $\{ Type \rightarrow \text{string} \} \cup$
...

Grammar modularization

C++ context

```
1 // Filetype: *.cpp
2 #include <string>
3 #include <vector>
4 #include "org/fizz.hpp"
5 #include "com/buzz.hpp"
6 using std::string;
7 // ...
8 int fizz() { return 3; }
9 string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12     string bang = "...";
13     □
14 }
```

Grammar contribution

$$\begin{aligned} &G_{\text{C++}} \cup \\ &G_{\text{<string>}} \cup \\ &G_{\text{<vector>}} \cup \\ &G_{\text{org/fizz.hpp}} \cup \\ &G_{\text{com/buzz.hpp}} \cup \\ &\{ \text{Type} \rightarrow \text{string} \} \cup \\ &\dots \\ &\{ \text{Int} \rightarrow \text{fizz()}, \text{S} \rightarrow \text{fizz();} \} \cup \end{aligned}$$

Grammar modularization

C++ context

```
1 // Filetype: *.cpp
2 #include <string>
3 #include <vector>
4 #include "org/fizz.hpp"
5 #include "com/buzz.hpp"
6 using std::string;
7 // ...
8 int fizz() { return 3; }
9 string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12     string bang = "...";
13     □
14 }
```

Grammar contribution

$G_{\text{C++}} \cup$
 $G_{\text{<string>}} \cup$
 $G_{\text{<vector>}} \cup$
 $G_{\text{org/fizz.hpp}} \cup$
 $G_{\text{com/buzz.hpp}} \cup$
 $\{ \text{Type} \rightarrow \text{string} \} \cup$
 $\dots$
 $\{ \text{Int} \rightarrow \text{fizz()}, S \rightarrow \text{fizz}(); \} \cup$
 $\{ \text{Str} \rightarrow \text{buzz(Str)}, S \rightarrow \text{buzz(Str)}; \} \cup$

Grammar modularization

C++ context

```
1 // Filetype: *.cpp
2 #include <string>
3 #include <vector>
4 #include "org/fizz.hpp"
5 #include "com/buzz.hpp"
6 using std::string;
7 // ...
8 int fizz() { return 3; }
9 string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12     string bang = "...";
13     □
14 }
```

Grammar contribution

$$\begin{aligned} &G_{\text{C++}} \cup \\ &G_{\text{<string>}} \cup \\ &G_{\text{<vector>}} \cup \\ &G_{\text{org/fizz.hpp}} \cup \\ &G_{\text{com/buzz.hpp}} \cup \\ &\{ \text{Type} \rightarrow \text{string} \} \cup \\ &\dots \\ &\{ \text{Int} \rightarrow \text{fizz()}, S \rightarrow \text{fizz}(); \} \cup \\ &\{ \text{Str} \rightarrow \text{buzz(Str)}, S \rightarrow \text{buzz(Str)}; \} \cup \\ &\dots \end{aligned}$$

Grammar modularization

C++ context

```
1 // Filetype: *.cpp
2 #include <string>
3 #include <vector>
4 #include "org/fizz.hpp"
5 #include "com/buzz.hpp"
6 using std::string;
7 // ...
8 int fizz() { return 3; }
9 string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12     string bang = "...";
13     □
14 }
```

Grammar contribution

$$\begin{aligned} &G_{\text{C++}} \cup \\ &G_{\text{<string>}} \cup \\ &G_{\text{<vector>}} \cup \\ &G_{\text{org/fizz.hpp}} \cup \\ &G_{\text{com/buzz.hpp}} \cup \\ &\{ \text{Type} \rightarrow \text{string} \} \cup \\ &\dots \\ &\{ \text{Int} \rightarrow \text{fizz()}, S \rightarrow \text{fizz}(); \} \cup \\ &\{ \text{Str} \rightarrow \text{buzz(Str)}, S \rightarrow \text{buzz(Str)}; \} \cup \\ &\dots \\ &\{ \text{Int} \rightarrow \text{argc}, \text{CharPtr} \rightarrow \text{args[Int]} \} \cup \end{aligned}$$

Grammar modularization

C++ context

```
1 // Filetype: *.cpp
2 #include <string>
3 #include <vector>
4 #include "org/fizz.hpp"
5 #include "com/buzz.hpp"
6 using std::string;
7 // ...
8 int fizz() { return 3; }
9 string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12     string bang = "...";
13     □
14 }
```

Grammar contribution

$$\begin{aligned} &G_{\text{C++}} \cup \\ &G_{\text{<string>}} \cup \\ &G_{\text{<vector>}} \cup \\ &G_{\text{org/fizz.hpp}} \cup \\ &G_{\text{com/buzz.hpp}} \cup \\ &\{ \text{Type} \rightarrow \text{string} \} \cup \\ &\dots \\ &\{ \text{Int} \rightarrow \text{fizz}(), S \rightarrow \text{fizz}(); \} \cup \\ &\{ \text{Str} \rightarrow \text{buzz}(\text{Str}), S \rightarrow \text{buzz}(\text{Str}); \} \cup \\ &\dots \\ &\{ \text{Int} \rightarrow \text{argc}, \text{CharPtr} \rightarrow \text{args}[\text{Int}] \} \cup \\ &\{ \text{Str} \rightarrow \text{bang} \} \rightarrow G_{\Gamma} \end{aligned}$$

Grammar modularization

C++ context

```
1 // Filetype: *.cpp
2 #include <string>
3 #include <vector>
4 #include "org/fizz.hpp"
5 #include "com/buzz.hpp"
6 using std::string;
7 // ...
8 int fizz() { return 3; }
9 string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12     string bang = "...";
13     □
14 }
```

Grammar contribution

$$\begin{aligned} &G_{\text{C++}} \cup \\ &G_{\text{<string>}} \cup \\ &G_{\text{<vector>}} \cup \\ &G_{\text{org/fizz.hpp}} \cup \\ &G_{\text{com/buzz.hpp}} \cup \\ &\{ \text{Type} \rightarrow \text{string} \} \cup \\ &\dots \\ &\{ \text{Int} \rightarrow \text{fizz()}, S \rightarrow \text{fizz}(); \} \cup \\ &\{ \text{Str} \rightarrow \text{buzz(Str)}, S \rightarrow \text{buzz(Str)}; \} \cup \\ &\dots \\ &\{ \text{Int} \rightarrow \text{argc}, \text{CharPtr} \rightarrow \text{args[Int]} \} \cup \\ &\{ \text{Str} \rightarrow \text{bang} \} \rightarrow G_{\Gamma} \\ &\mathcal{L}(G_{\Gamma}) \end{aligned}$$

Nominal typing judgements

Context

Γ contains all declarations visible at the cursor

$\Gamma \vdash e : T$

C++ accepts e and gives it the concrete type T .

C++ program

```
struct Meter {  
    int read(int n) { return n + 1; }  
};  
int main() {  
    Meter m;  
    int n = 2;  
    int x = m.read(n);  
    return x + x;  
}
```

Embedding

$\Gamma \vdash m.read(n) : int$

$\Downarrow$

$E\langle int \rangle \rightarrow E\langle Meter \rangle.read(E\langle int \rangle)$

$\Gamma \vdash x + x : int$

$\Downarrow$

$E\langle int \rangle \rightarrow E\langle int \rangle + E\langle int \rangle$

Embedding Substitute locally bound names with typed slots.

Subtyping: tracking inheritance

Derived-to-base call

`Dog <: Animal`

`Dog` publicly inherits `Animal`, so `d` may be passed to `age` as an `Animal`.

C++ program

```
struct Animal {};  
struct Dog : Animal {};  
int age(Animal) { return 7; }  
int main() {  
    Dog d;  
    int x = age(d);  
    return x;  
}
```

Embedding

$$\Gamma \vdash \text{age} : \text{Animal} \rightarrow \text{int}$$
$$\Downarrow$$
$$E\langle \text{int} \rangle \rightarrow \text{age}(E\langle \text{Dog} \rangle)$$
$$E\langle \text{int} \rangle \rightarrow \text{age}(E\langle \text{Animal} \rangle)$$

`age` is contravariant in the input.

Keep the check local

Only track well-scoped subtypes and embed each subtype monomorphically.

Templates: try concrete types, keep valid rules

C++ program

```
#include <string>
#include <vector>
using std::string;
template<class T>
T twice(T x) { return x + x; }
int main() {
    int i = 2; int n = twice(i);
    string h = "ha"; string s = twice(h);
    return n == 4 || s == "haha";
}
```

Try three concrete types

int ✓ accepted

std::string ✓ accepted

std::vector<int> ✗ rejected

The vector case fails because $x + x$ is not defined for vectors.

Keep the successful rules

$$\Gamma \vdash \text{twice}(i) : \text{int} \implies E(\text{int}) \rightarrow \text{twice}(E(\text{int}))$$
$$\Gamma \vdash \text{twice}(h) : \text{std::string} \implies E(\text{std::string}) \rightarrow \text{twice}(E(\text{std::string}))$$

Sound: every kept rule compiles.

Complete within this set: all three were tried.

Class templates: invariant by default

Subtyping does not lift automatically

`Dog <: Animal` $\nRightarrow$ `F<Dog> <: F<Animal>`

Class templates may optionally have a constrained converting constructor.

API-provided covariance

```
std::shared_ptr<Dog> pd;  
std::shared_ptr<Animal> pa = pd;  
// accepted
```

`shared_ptr` explicitly supplies the conversion.

Invariant specialization

```
std::vector<Dog*> ds;  
std::vector<Animal*> as = ds;  
// rejected
```

The two `vector` specializations are unrelated.

CFG consequence

enumerate ground states; validate exact contexts

Never infer that `F<D>` converts to `F<B>` merely from `D <: B`. Emit a rule only when the C++ API supplies and accepts that conversion.

Results: Size of generated CFGs

Evaluation